

Lecture: 0110

Registers, Counters & State Machines - II

Building Memory That Does Something

Chain flip-flops together and a circuit can store words, count, and follow a plan.

CSA222: Modern Computer Architecture

SECTION 02

2. Counters

The Beating Clock of Digital Systems

Flip-flops configured to count clock pulses, serving as the essential foundation for timing, addressing, and control flow.

CORE APPLICATIONS

**Timers & Clock Dividers**

Measuring precise time intervals and scaling frequencies.

**Address Generation**

Stepping through memory locations sequentially.

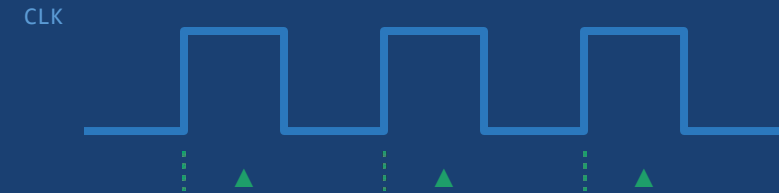
**The Program Counter (PC)**

The engine driving instruction fetch and CPU execution flow.

VISUALIZING CLOCK & COUNT

How a Counter Advances

Each rising edge of the clock trigger increments the register state.

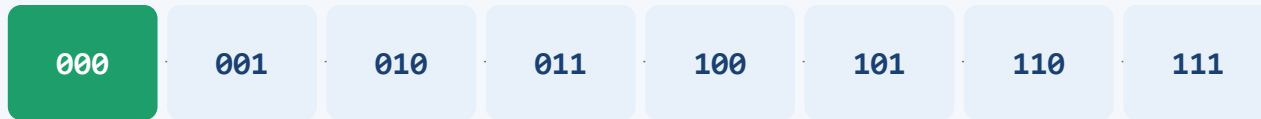


STATE TRANSITION (N-BIT OVERFLOW)

What Is a Counter?

A register whose value advances by 1 on every clock pulse — it counts.

A 3-BIT COUNTER CYCLES THROUGH 0 → 7 → 0...



↻ wraps back to 000

Each clock pulse = +1.

After the max value (111 = 7), it rolls over to 000 — exactly like an odometer, or 8-bit overflow from L4.

An n -bit counter counts 0 to 2^n-1 , then wraps. It's a register + an incrementer + feedback.

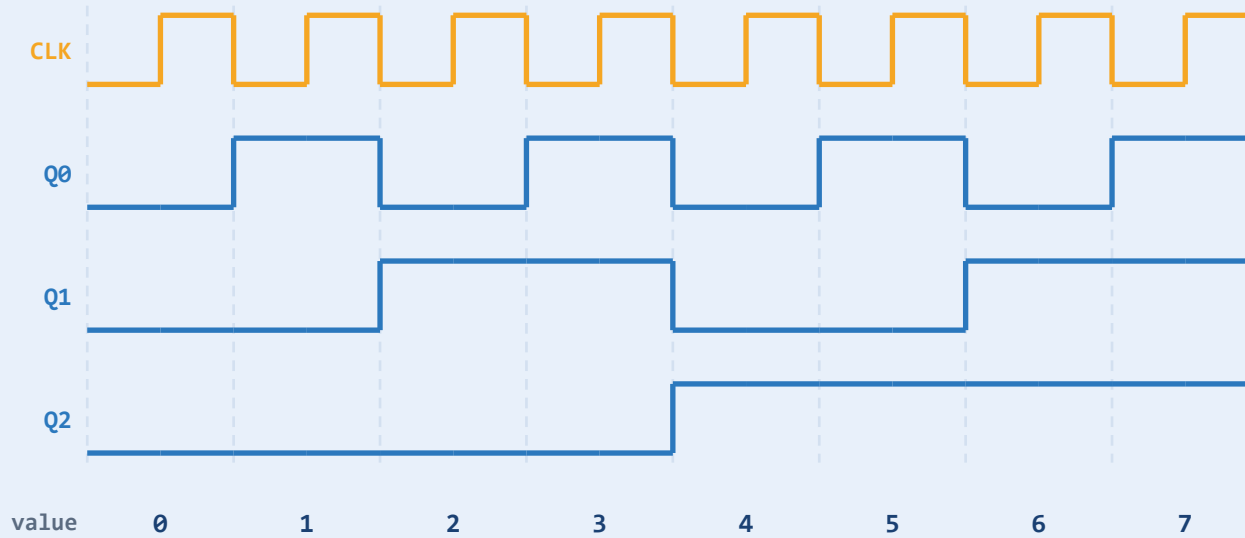
WHY COUNTERS MATTER

The Program Counter (PC) is a counter — it holds the address of the next instruction and increments each cycle.

Also: timers, clock dividers, memory-refresh logic, event counters — all built on this.

Reading the Count as Waveforms

Look at the bits over time and the pattern that makes counters buildable jumps out.



Q0 changes every clock. Q1 changes every second clock. Q2 changes every fourth.

THE RULE BEHIND IT

A bit flips exactly when every bit below it is 1.

Q0 flips always. Q1 flips when Q0 = 1. Q2 flips when Q1·Q0 = 1.

IN T FLIP-FLOP TERMS

$T0 = 1$

$T1 = Q0$

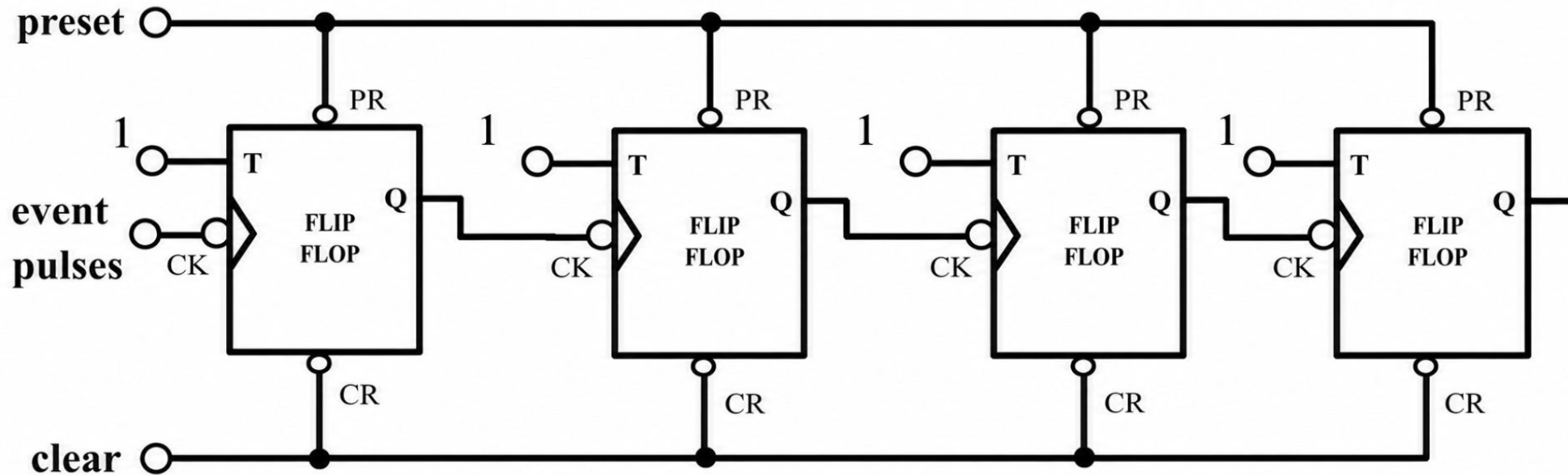
$T2 = Q0 \cdot Q1$

THAT IS THE WHOLE DESIGN

Nothing else is needed for a plain binary up-counter. Two different circuits implement it — and choosing between them is the next four slides.

Ripple (Asynchronous) Counter

Each flip-flop's output clocks the next — simple, but the carry ripples.



Only FF0 sees the real clock; each next FF is clocked by the previous output — the change 'ripples' along.

⚠ The ripple delay means the count is briefly **WRONG** while it propagates — a problem at high speed.

Why Ripple Counters Break at Speed

Follow the count from 7 to 8 one gate delay at a time, and watch three wrong numbers appear on the pins.

TIME AFTER EDGE	Q3	Q2	Q1	Q0	VALUE ON PINS
before	0	1	1	1	7 — correct
1 × t _{pd}	0	1	1	0	6 — wrong
2 × t _{pd}	0	1	0	0	4 — wrong
3 × t _{pd}	0	0	0	0	0 — wrong
4 × t _{pd}	1	0	0	0	8 — correct at last

Anything reading those pins during the ripple — a comparator, a decoder, a memory address bus — sees 6, then 4, then 0.

THE SPEED LIMIT

The count is only valid after ALL n stages have settled:

$$t_{\text{settle}} = n \times t_{\text{pd}}$$

$$f_{\text{max}} = 1 / (n \times t_{\text{pd}})$$

PUT NUMBERS ON IT

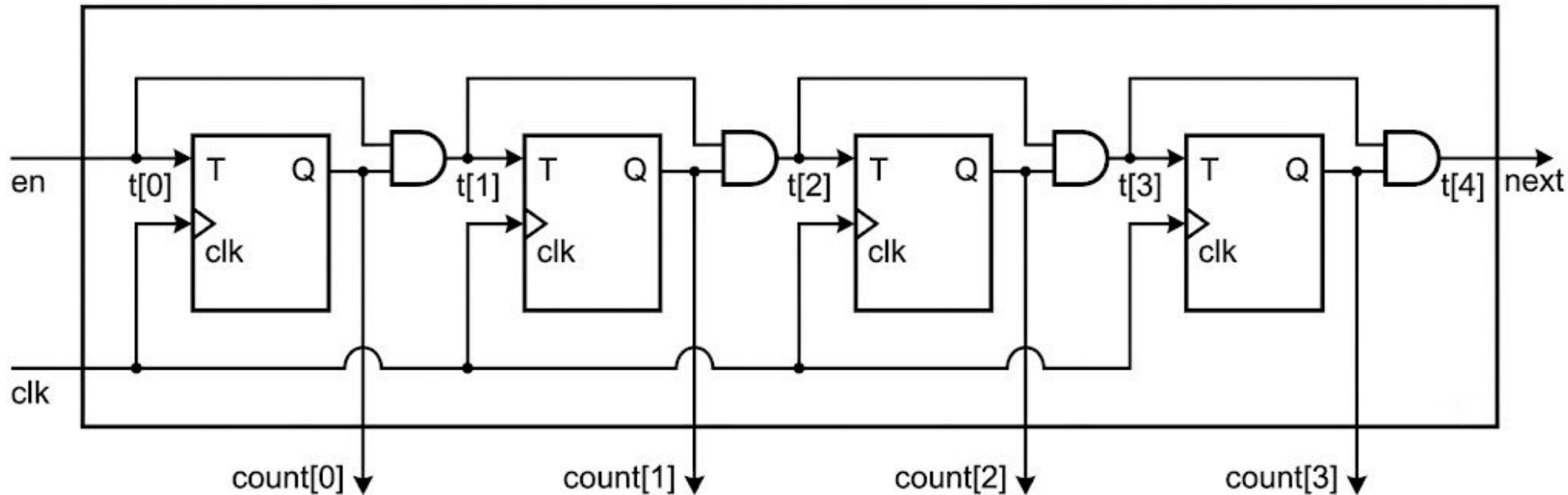
At $t_{\text{pd}} = 10 \text{ ns}$, a 4-bit ripple counter tops out near 25 MHz — and a 16-bit one near 6 MHz. A modern CPU clock is 3 GHz.

THE WAY OUT

Give every flip-flop the same clock, and compute in advance which ones should flip.

Synchronous Counter

All flip-flops share ONE clock — they all update at the exact same instant.



How It Works

Combinational logic (the & gates) computes each FF's next bit; the shared clock updates them all together.

Key Advantage

✓ No ripple delay — the whole count is correct the instant after the edge. This is what real designs use.

Ripple vs Synchronous

Same count sequence, very different timing behaviour.

	RIPPLE (async)	SYNCHRONOUS
Clocking	Each FF clocks the next	All FFs share one clock
Speed	Slow — carry ripples through	Fast — all update at once
Glitches	Yes — transient wrong counts	No — clean output
Complexity	Simple, fewer gates	Needs next-state logic
Real use	Toy / low-speed dividers	Everything performance-critical

Trade-off in one line: ripple is cheaper to build; synchronous is the only one safe at real clock speeds.

Mod-N & Divide-by Counters

Reset early to count any range — or use a counter to slow a clock down.

MOD-N: STOP EARLY

A 3-bit counter naturally counts mod-8 (0–7). To count mod-6 (0–5), detect the value 6 and reset to 0.



↻ reset at 6

Used for: clock dividers, digital clocks (mod-60 for seconds!), any custom cycle length.

DIVIDE-BY: SLOW THE CLOCK

Each flip-flop in a counter toggles at HALF the rate of the one before it.

So bit 0 = clock ÷ 2, bit 1 = clock ÷ 4, bit 2 = clock ÷ 8...

A counter is a frequency divider for free. This is how a 32 kHz crystal becomes a 1 Hz tick in a watch.

Counters in the Wild

Once you see them, they're everywhere.



Program Counter

Holds the address of the next instruction; increments each cycle, or jumps on a branch. The heart of instruction flow.



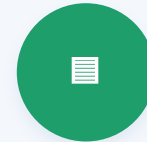
Timers & delays

Count clock ticks to measure time — every microcontroller timer, watchdog, and PWM generator is a counter.



Frequency division

Turn a fast crystal oscillator into slower usable clocks — watches, UART baud rates, display refresh.



Memory addressing

Sequentially step through addresses in DMA transfers, refresh cycles, and scan-out logic.

3. Finite State Machines

The universal model of sequential logic: a set of states, rules to move between them, and outputs. This is how a CPU 'thinks'.

What Is a Finite State Machine?

A circuit that is always in exactly ONE of a finite set of states, and moves between them on rules.

THREE INGREDIENTS

1

States

A finite list of situations the machine can be in (e.g. IDLE, RUNNING, DONE).

2

Transitions

Rules for moving between states, based on current state + inputs.

3

Outputs

What the machine produces in each state (or during each transition).

EVERYDAY EXAMPLE

A traffic light is an FSM.

States: GREEN, YELLOW, RED.

Transitions: timed (green→yellow→red→green).

Outputs: which lamp is lit.

A vending machine, an elevator, a game character's AI, a network protocol — all FSMs. This model is everywhere.

Two Flavours: Moore vs Mealy

The difference is simply WHERE the output comes from.

MOORE MACHINE

Output depends on the STATE only

The output is written inside each state circle

It changes only when the state changes

Simpler to reason about, and glitch-free

May need more states to do the same job

Output appears one cycle after the input

MEALY MACHINE

Output depends on STATE + INPUT

The output is written on each transition arrow

It can react to an input within the same cycle

Often needs fewer states

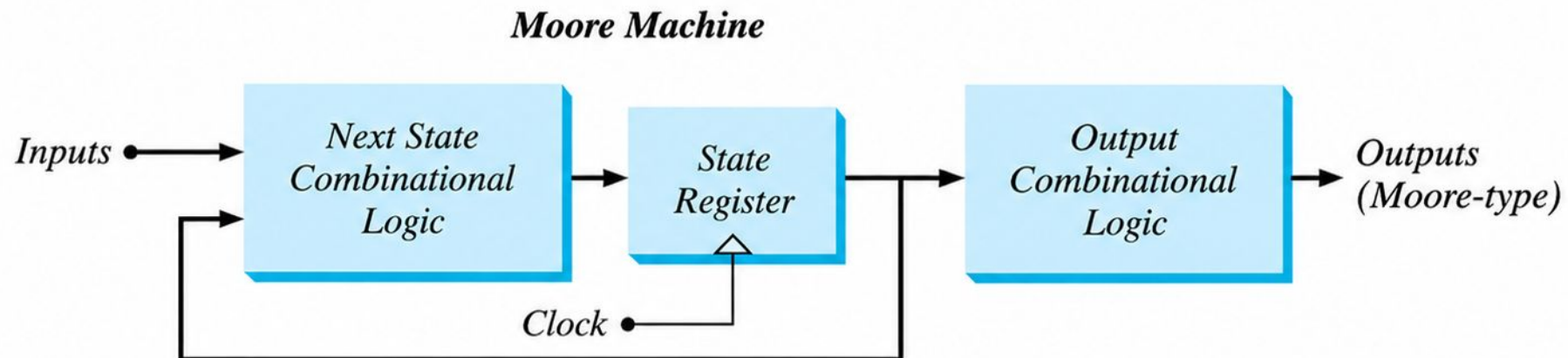
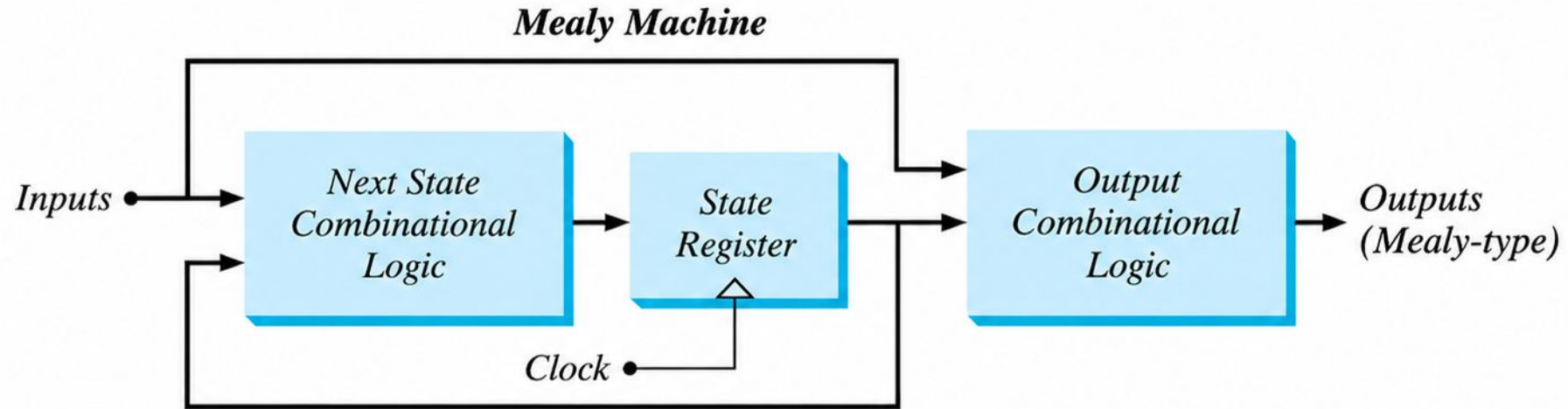
Output can glitch when the input glitches

Output appears in the same cycle as the input

Rule of thumb: Moore = 'output in the circle', Mealy = 'output on the arrow'. Both compute the same things.

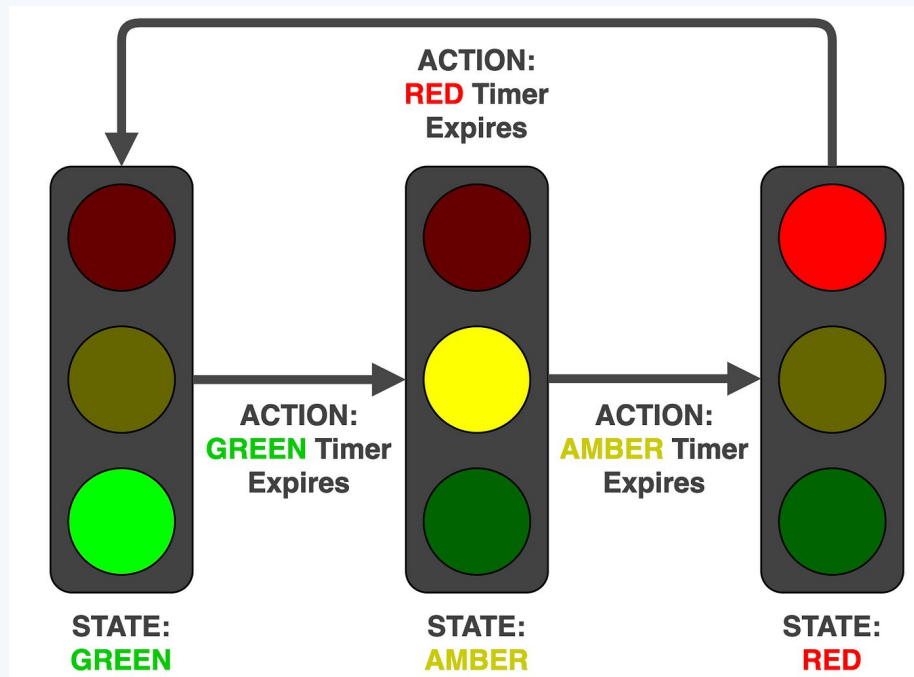
The Anatomy of Any FSM

Every sequential circuit is this same three-part structure.



The State Diagram

The picture that captures an FSM: circles are states, arrows are transitions.



HOW TO READ IT

Each circle = one state.

Each arrow = a transition, labelled with the input/condition that triggers it.

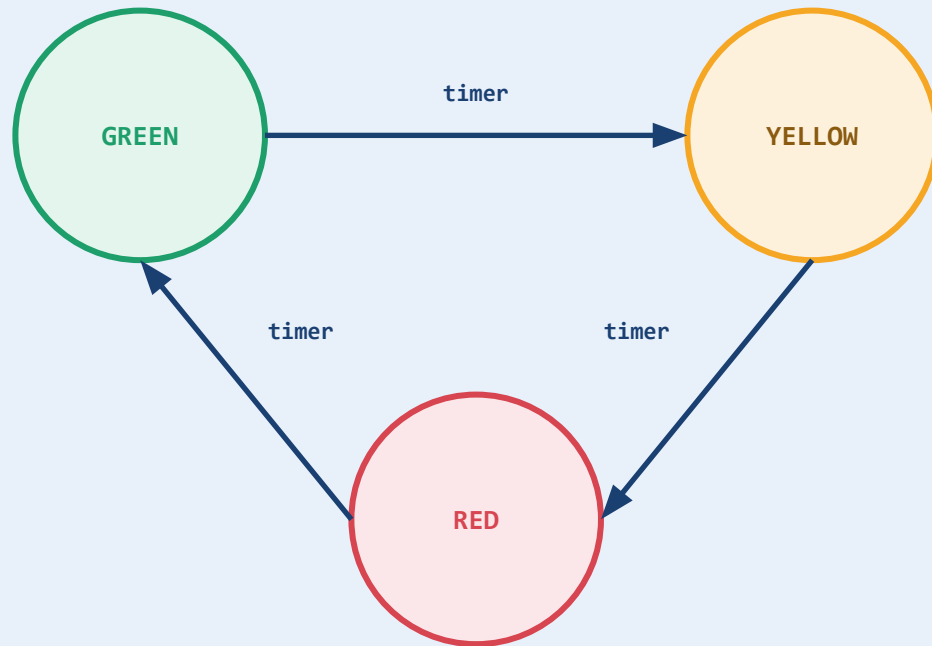
Follow the arrows and you can trace exactly what the machine does for any input sequence.

This diagram IS the design — everything else is just translating it to gates.

The State Diagram

The picture that captures an FSM: circles are states, arrows are transitions.

TRAFFIC LIGHT — A THREE-STATE MOORE MACHINE



HOW TO READ IT

Each circle = one state.

Each arrow = a transition, labelled with the input/condition that triggers it.

Follow the arrows and you can trace exactly what the machine does for any input sequence.

This diagram IS the design — everything else is just translating it to gates.

The State Table

The same FSM, written as a table — the bridge from diagram to hardware.

Current State	Input	Next State	Output
GREEN	expired	YELLOW	Go
YELLOW	expired	RED	Slow
RED	expired	GREEN	Stop
any	not expired	(same)	(hold)

Why a table? It lists every (state, input) → (next state, output). Encode the states as bits, and each column becomes a truth table you minimise with K-maps (L2) into gates.

FSM Design Flow

Six steps from a word problem to a working circuit — always the same recipe.

1**Understand**

List the states and what triggers each transition.

2**State diagram**

Draw circles and arrows — the visual spec.

3**State table**

Tabulate every state/input → next-state/output.

4**Encode states**

Assign a bit pattern to each state (00, 01, 10...).

5**Derive logic**

K-map each next-state & output bit into gates.

6**Build**

Flip-flops for the state register + that logic.

FSMs Run the World

Once you know the pattern, you'll see state machines in everything you build.



The CPU control unit

Fetch → Decode → Execute → Writeback is literally an FSM. It sequences every instruction (Module 4).



Communication protocols

TCP connection states, USB handshakes, traffic-light controllers — all specified as FSMs.



Software & games

UI flows, game AI ('patrol → chase → attack'), regex engines, parsers — FSMs in code, not gates.

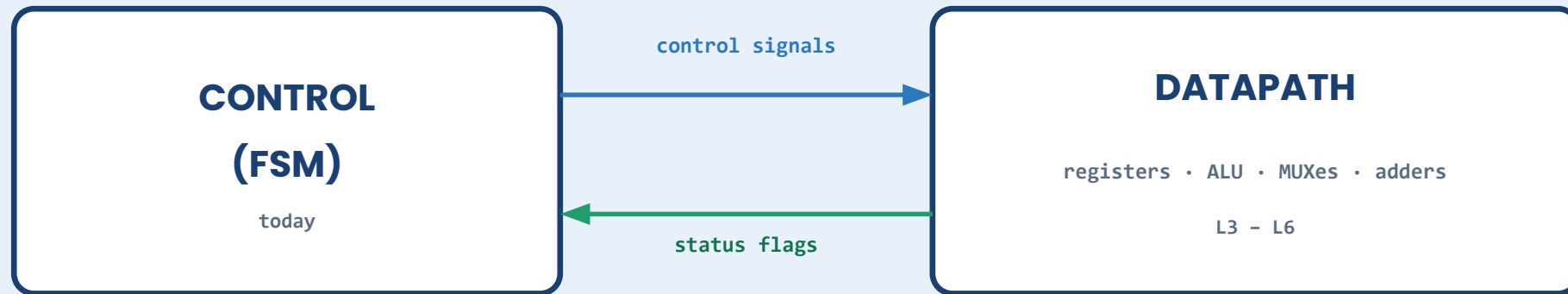


Embedded control

Washing machines, elevators, vending machines, engine controllers — physical FSMs everywhere.

The Big Picture: Datapath + Control

Everything in this course so far now snaps together into a computer.



The control unit is an FSM (today). The datapath is registers + ALU + MUXes (L3-L6). Together they're a CPU.

This is exactly where Module 3 & 4 pick up — you now have every building block a processor needs.

Common Mistakes to Avoid

The slips that cost marks — and cause real design bugs.



Register without load enable

A plain register overwrites every clock. Real ones need a load/hold control, or they can't keep a value.



Moore/Mealy confusion

Moore output = state only (in the circle). Mealy = state + input (on the arrow). Know which you're drawing.



Ripple counter at high speed

The carry ripple makes intermediate counts briefly wrong. Use synchronous counters for anything fast.



Forgetting the reset state

Every FSM needs a defined power-on state, or it starts in a random, possibly illegal state.



Wrong select/state-bit count

N states need $\lceil \log_2 N \rceil$ flip-flops (binary). Miscounting breaks the whole encoding.



Unreachable / dead states

After encoding, unused bit patterns can trap the FSM. Handle them (usually route to reset).

Key Takeaways

Five sentences — from a stored bit to a thinking machine.

- 01** A register is n flip-flops sharing a clock — the CPU's fastest storage; add a load-enable MUX so it holds until told to update.
- 02** A shift register moves bits along a chain each clock — enabling serial↔parallel conversion and free multiply/divide-by-2.
- 03** A counter advances by 1 each clock; synchronous (shared clock) beats ripple (chained) for speed and glitch-free counts.
- 04** A finite state machine — states, transitions, outputs — is the universal model; registers and counters are special cases of it.
- 05** FSM design is a fixed recipe: diagram → table → encode states → K-map the logic → build with a state register + gates.

Practice & Homework

Design, trace, and build state machines.

01

Shift & Count

Trace a 4-bit shift register loading 1011 over 4 clocks. Then draw a 3-bit synchronous counter's Q outputs.

02

FSM from Words

Design a '110' sequence detector: state diagram, state table, and binary encoding.

03

Divide the Clock

How many flip-flops divide a 1 MHz clock down to 1 kHz? Show the divide-by math.

EXIT TICKET

1. How many flip-flops does a mod-12 counter need, and why?
2. In one line: the difference between a Moore and a Mealy machine.
3. Draw the state diagram for a circuit that detects the input sequence '11'.

Rapid-Fire Recap

Three questions — shout the answers.

Q1

A register is built from...

flip-flops

Q2

Synchronous counters share one...

clock

Q3

A Moore output depends only on...

the state

All three instant? You've completed the digital-logic foundation — Module 3 builds the datapath on top of it.

END OF L06 • MODULE 2 COMPLETE

States, and a machine that runs.

NEXT — MODULE 3

The Datapath & Instruction Set — assemble registers, ALU, and control into a processor that actually executes machine instructions.



**Thanks
for
watching!**